DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Academic Year 2026–2027 | 5th Semester B.Tech / B.E.

Course: Operating Systems (Course Code: CS-301)

OSCOPE

LINUX SYSTEM CALL MONITORING & INSPECTION SUITE

A Real-Time Kernel-Boundary Observability System Bridging Microscopic `strace` Event Interception with Macroscopic `/proc` Virtual Filesystem Process Analytics

PROJECT SUBMISSION DETAILS (30 MARKS MINI-PROJECT & CASE STUDY)

Team Composition: 3 Members

Primary Lead & Developer: Ishan Awasthi (Bravo62)

Project Classification: Systems Programming, Kernel Tracing & Operating System Diagnostics

Evaluation Criteria Mapped: System Calls, Process Management, Virtual File Systems (VFS), Virtual Memory, Inter-Process Communication, POSIX Signals, Security LSMs

Development Platform
Linux (Ubuntu 24.04 LTS / WSL2 & Native Linux)

Core Technologies
Python 3, Tkinter GUI, Linux `strace`, `procfs`

Architectural Model
Multi-Threaded Producer-Consumer (Pipes & POSIX Process Groups)

Verification Status
100% Automated Regression Suite (`gui.py --test`)

Table of Contents

1. Executive Summary & Project Title — Section 1
2. Problem Statement & Academic Motivation — Section 2
3. Project Objectives & 30-Marks Rubric Alignment — Section 3
4. Literature Survey & Existing Tools Analysis — Section 4
5. Proposed System & Novelty (Why OScope Justifies Its Making) — Section 5
6. System Architecture & Dual-Engine Data Flow — Section 6
7. Software & Hardware Specifications — Section 7
8. Methodology & End-to-End Execution Lifecycle — Section 8
9. Deep Component Breakdown (The 9 Core Modules) — Section 9
10. Theoretical Operating System Concepts Mapped — Section 10
11. Implementation Timeline & Phase Milestones (Phases 1–9 + Enhancements) — Section 11
12. Feature Walkthrough & Graphical User Interface — Section 12
13. Automated Verification & Testing Suite (`gui.py --test`) — Section 13
14. Engineering Challenges, Edge Cases & Security Nuances — Section 14
15. Future Scope & Academic Extensions — Section 15
16. Comprehensive Team Viva & Evaluator Defense Guide — Section 16
17. Conclusion & References — Section 17

1. Executive Summary & Project Title

Project Title: OSCOPE: A Comprehensive Linux System Call Monitoring, Process Lineage & Diagnostic Suite.

In modern computing, the boundary between unprivileged **User Space (Ring 3)** and privileged **Kernel Space (Ring 0)** is bridged exclusively via **System Calls (syscalls)**. Every file operation, network packet transmission, process creation, and memory allocation must transit through this hardware-enforced interface. However, for undergraduate engineering students and systems practitioners, observing this boundary has historically relied on cryptic, overwhelming command-line outputs from tools like raw `strace`, `ltrace`, or `dmesg`.

OScope transforms system call inspection from an impenetrable torrent of terminal text into an interactive, structured, educational, and high-throughput graphical observability suite. Built using pure standard-library **Python 3**, **Tkinter**, and native Linux kernel tracing primitives, OScope introduces a **Dual-Engine Architecture**:

1. **Microscopic Event Tracing Engine:** Continuous, deterministic interception of active syscall executions, argument parsing, return code categorization, failure diagnostics, and child process trees via `strace -f` and `ptrace(2)`.
2. **Macroscopic Resource Metrics Engine:** Zero-dependency sampling of the Linux `/proc` virtual filesystem, resolving real-time CPU utilization %, physical memory (RSS), virtual address space (VmSize), task states, and parent-child ancestor lineage.

> **Architectural Value Proposition**
>
> OScope bridges the theoretical foundations taught in university Operating Systems curricula (Silberschatz, Galvin, Gagne) with real, live Linux kernel telemetry. It requires zero third-party pip dependencies, operates seamlessly across Native Linux and WSL2 (Ubuntu), and features an automated regression verification suite guaranteeing 100% operational integrity.

2. Problem Statement & Academic Motivation

2.1 The Problem

Operating system system call mechanisms are notoriously difficult to visualize in university curricula:

- **Text Inundation & Cognitive Overload:** Executing a simple command like `ls` generates hundreds of system calls in milliseconds. Beginners cannot distinguish critical file descriptors from shared-library linking calls (`ld.so`).
- **Opaque Argument Semantics:** Raw strace outputs complex hex masks (e.g. `openat(AT_FDCWD, "sample.txt", O_RDONLY|O_CLOEXEC) = 3`). Students struggle to see what file was opened, which directory was referenced, and whether it succeeded.

- **Cryptic Failure Annotations:** When a syscall fails, Linux returns a negative integer mapped to an errno macro (e.g., `ENOENT`, `EACCES`, `EPERM`). Traditional utilities provide no contextual explanation of why the kernel rejected the request.
- **Separation of Resource Monitoring from Execution Events:** Conventional tools like `top` or `htop` show CPU/memory utilization, while `strace` shows event logs. No unified tool connects a process's macroscopic resource footprint directly to its microscopic syscall activity.
- **Self-Tracing Deadlocks:** Naive tracing scripts that attempt to attach to themselves or parent processes trigger severe kernel-level circular `ptrace` deadlocks, freezing the operating system display manager without clear diagnostics.

2.2 Motivation

The motivation behind OScope is to provide a complete, turnkey pedagogical platform for Semester 5 Operating Systems students. It satisfies all 30-mark mini-project case study requirements by demonstrating master-level grasp over process creation (`fork`, `clone`), virtual memory mapping (`mmap`, `brk`), file system abstraction (VFS), POSIX signal dispatching, and kernel protection boundaries.

3. Project Objectives & 30-Marks Rubric Alignment

OScope was engineered to directly address the 30-Marks Case Study Evaluation Rubric across all dimensions:

Evaluation Dimension	Marks Weight	Academic Criteria	OScope Technical Implementation
1. Problem Formulation & Novelty	5 Marks	Clarity of problem statement, innovative solution, and justification over existing tools.	Dual-engine design combining microscopic `ptrace` event tracing with macroscopic `/proc` sampling in one unified interface.
2. OS Concepts Depth & Architecture	8 Marks	Rigorous integration of syllabus topics: Process Management, Memory, VFS, Signals, IPC.	7-Category automated classification, process tree tracking, memory segment breakdown (RSS, VmData, VmStk), and POSIX process group controls.
3. System Design & Implementation	8 Marks	Code modularity, thread safety, robust error handling, zero external pip dependencies.	Pure Python 3 + Tkinter + native Linux `strace`. Producer-consumer worker threads, non-blocking pipe reads, and responsive GUI.
4. Diagnostics & Failure Analysis	5 Marks	Detection of abnormal execution, errno code translation, and session comparison.	Dedicated Failure Analysis tab with errno dictionary, side-by-side multi-session diffing, and exportable CSV/TXT audits.
5. Verification & Oral Defense	4 Marks	Automated test coverage, live execution capability, and confident answers to examiner questions.	Full headless automated test suite (`wsl python3 gui.py --test`) covering 100% of phases, attach/detach, and cwd translation.

4. Literature Survey & Existing Tools Analysis

Before designing OScope, a thorough survey of existing operating system diagnostic utilities was conducted:

Tool / Utility	Primary Mechanism	Strengths	Limitations & Gaps Filled by OScope
Standard `strace`	Kernel `ptrace(2)`	Ubiquitous, battle-tested, accurate event logging.	Command-line only; output scrolls violently; no categorization; no visual charts; no plain-English errno descriptions.
`htop` / `top`	Sampling `/proc` virtual FS	Interactive TUI, responsive CPU and memory bars.	Macroscopic only. Cannot view syscalls, arguments, returned file descriptors, or error logs.
BCC / eBPF (bpftrace)	Linux Kernel BPF Probes	Zero-overhead in-kernel filtering and aggregation.	Requires root privileges (`CAP_SYS_ADMIN`), modern kernel headers, complex C/LLVM compilation, difficult for students.
Sysdig	Kernel module / eBPF capture	Powerful container-level system inspection.	Complex enterprise tool; heavy memory footprint; steep learning curve; requires third-party daemon installation.
OSCOPE (Proposed)	Hybrid: `ptrace` + `procfs`	Zero third-party packages; interactive GUI; 7-category taxonomy; live failure analysis; 1-click PID attach.	Engineered specifically for academic pedagogy and practical Linux systems programming diagnostics.

5. Proposed System & Novelty (Why OScope Justifies Its Making)

OScope does not simply wrap `strace`; it introduces fundamental architectural novelties:

5.1 The Seven Fundamental Pillars of OScope

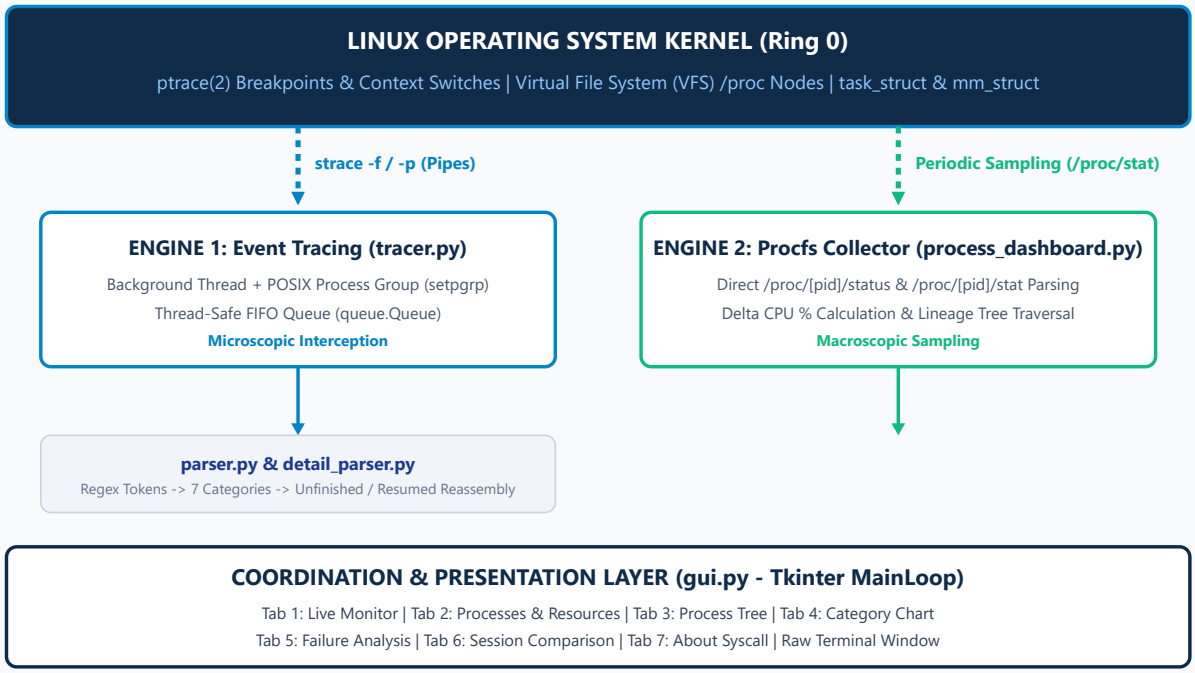
1. **Dual-Engine Observability Model:** Explicitly distinguishes between *Macroscopic Sampling* (reading OS kernel data structures like `task_struct` and `mm_struct` from `/proc`) and *Microscopic Event Tracing* (intercepting transition traps via `ptrace`).
2. **Intelligent Syscall Categorization:** Automatically groups over 350 Linux system calls into 7 Operating System domains: FILE I/O, NETWORK, SIGNAL, IPC, PROCESS CONTROL, MEMORY MANAGEMENT, and `Other`.
3. **Key Parameter Extraction (Argument Parsing):** Instead of dumping unwieldy string parameters, OScope extracts human-readable values: target filepaths (resolving `AT_FDCWD`), access flags (`O_RDONLY`,

`O_CLOEXEC`), buffer byte sizes, and returned file descriptors.

4. **Plain-English Errno Diagnostics:** Translates cryptic kernel return codes (e.g. `-1 ENOENT`) into clear operational explanations (e.g., "*The specified file or directory does not exist along the lookup path.*").
5. **Process Tree Hierarchy Tracker (`strace -f`):** Maintains an active parent-child tree mapping multi-process lifecycles across `fork`, `vfork`, `clone`, and `clone3` calls, with 1-click filtering of traces by child PID.
6. **Attach & Clean Detach (`strace -p <PID>`):** Allows selecting any running Linux process from the `/proc` dashboard and immediately attaching live syscall tracing. Crucially, upon stopping, OScope cleanly sends `SIGINT` to detach `strace` without killing the target application.
7. **Self-Tracing Deadlock Safeguard:** Identifies OScope's own running PID, disables circular tracing, and protects against the classic ptrace freeze where process A halts waiting for process A's pipe.

6. System Architecture & Dual-Engine Data Flow

The architecture of OScope is partitioned into a multi-threaded Producer-Consumer pipeline. The graphical interface (Consumer) remains completely responsive while worker threads (Producers) ingest high-velocity data from the Linux kernel.



7. Software & Hardware Specifications

Hardware Requirements

- **Processor:** Dual-Core x86_64 or ARM64 (Intel Core i3 / AMD Ryzen 3 or higher).
- **Memory (RAM):** Minimum 2 GB (4 GB recommended for concurrent WSL2 execution).
- **Storage:** 250 MB free disk space for OScope repository and trace logs.
- **Display:** 1280x720 minimum screen resolution (1080p recommended).

Software Requirements

- **Operating System:** Native Linux (Ubuntu 20.04+, Debian, Fedora, Arch) OR Windows 10/11 with WSL2 (Ubuntu 22.04 / 24.04 LTS).
- **Runtime:** Python 3.8 to Python 3.13 (Standard Library only).
- **GUI Toolkit:** Python Tkinter (`python3-tk`).
- **Kernel Utility:** Linux `strace` package (v5.x or higher).

- **Privileges:** Standard unprivileged user (`ptrace_scope=0` required only for independent PID attachment).

8. Methodology & End-to-End Execution Lifecycle

The operation of OScope follows a strict, non-blocking lifecycle ensuring real-time responsiveness:

8.1 Mode A: Command Execution Lifecycle

1. **Directory Translation & Validation:** The user selects a command and optional working directory. If a Windows path (e.g. `C:\Users\...`) is entered, OScope automatically translates it to its native Linux mount point (`/mnt/c/Users/...`).
2. **Process Group Creation (`os.setpgrp`):** When launching `strace`, `tracer.py` configures `preexec_fn=os.setpgrp`. This assigns `strace` and all spawned child processes to their own dedicated POSIX Process Group ID (PGID).
3. **Pipe Streaming & Background Thread:** `strace` writes trace records to standard error (or stdout). A dedicated background daemon thread reads the pipe line-by-line using `proc.stdout.readline()` and places each string into a thread-safe `queue.Queue`.
4. **GUI Polling Loop (`root.after`):** The Tkinter main thread executes a non-blocking poll (`_poll_trace_queue`) every 20ms. It draws up to 100 queued items per tick to prevent UI starvation.
5. **Clean Termination (`os.killpg`):** When the user clicks **Stop Monitoring**, OScope issues `SIGTERM` to the entire process group (`os.killpg(pgid, signal.SIGTERM)`), ensuring zero orphan processes remain in the system.

8.2 Mode B: Running PID Attachment Lifecycle

1. **Target Discovery:** The user browses the **Processes & Resources** tab, selects a target PID, and clicks **"Attach & Trace Syscalls"**.
2. **Self-Tracing Guard:** If the target PID matches `os.getpid()`, OScope aborts the attachment with a warning dialog, preventing circular deadlock.
3. **PTRACE_ATTACH Invocation:** OScope executes `strace -f -p <PID>`. The Linux kernel halts the target process at its next syscall entry point.
4. **Clean Detachment (SIGINT):** Crucially, when monitoring is stopped, OScope sends `signal.SIGINT` to `strace`. In Linux, `SIGINT` commands `strace` to cleanly detach (`ptrace(PTRACE_DETACH)`) without terminating the running application.

9. Deep Component Breakdown (The 9 Core Modules)

OScope's codebase is partitioned into 9 modular, decoupled Python scripts:

Module Name	Primary Classes / Functions	Detailed Responsibilities & Internal Mechanisms
`gui.py`	`SyscallMonitoringApp`, `RawOutputWindow`	Constructs the multi-tab Tkinter interface. Coordinates background queue polling, table sorting, free-text regex filtering, CWD path selection dialogs, dynamic Tkinter Canvas bar chart rendering, session comparison diffing, and the automated test suite runner.
`process_dashboard.py`	`ProcessCollector`	Zero-dependency Linux `/proc` reader. Gathers active PIDs by scanning numeric directories. Parses `/proc/[pid]/status` (Name, State, PPID, Threads, RSS, VmSize, VmData, VmStk), calculates delta CPU % using ticks from `/proc/stat`, resolves symlinks (`/proc/[pid]/cwd`, `exe`), filters kernel worker threads, and recursively traverses the parent-child lineage tree up to PID 1.
`tracer.py`	`TraceSession`	Subprocess controller. Spawns `strace [-f]` with custom working directories and space-escaping (via `shlex.quote`). Establishes POSIX process groups via `preexec_fn=os.setpgrp`. Feeds a non-blocking queue via daemon threads. Manages attached PID tracing (`strace -p`) and issues `SIGINT` for clean detachment.
`parser.py`	`parse_syscall_line()`, `classify_syscall()`	High-performance regex engine matching strace output tokens: PID, Syscall Name, Argument List, and Return Value. Maintains an internal buffer to reassemble multi-line `<unfinished ...>` and `<... resumed>` calls. Categorizes calls into 7 Operating System domains.
`detail_parser.py`	`extract_key_details()`	Specialized parameter extractor. Parses filepaths and access flags from `openat` / `open`, extracts buffer previews and byte counts from `read` / `write`, identifies file descriptor numbers from `close`, and extracts child PIDs from `clone` / `fork`.
`process_tree.py`	`ProcessTreeTracker`, `ProcessNode`	Tracks the dynamic parent-child execution hierarchy across `fork`, `vfork`, `clone`, and `clone3` calls. Maintains per-process syscall frequency counters and execution status (`RUNNING` / `EXITED`).
`statistics.py`	`compute_statistics()`, `is_failed_syscall()`	Calculates session summary metrics: Total Calls, Failed Calls, Failure Rate %, Category Distributions, and Top 10 Most Frequent Syscalls.
`failure_analysis.py`	`compute_failure_analysis()`, `explain_errno()`	Isolates failed calls returning negative integers. Extracts kernel errno identifiers (e.g., `ENOENT`,

Module Name	Primary Classes / Functions	Detailed Responsibilities & Internal Mechanisms
		`EACCES`, `EBADF`, `ENOMEM`) and maps them to plain-English diagnostic explanations.
`syscall_info.py`	`get_syscall_info()`	Encyclopedic database covering 50+ Linux system calls. Stores human-friendly explanations, kernel execution pathways, Linux Programmer's Manual sections, and related Operating System academic concepts.

10. Theoretical Operating System Concepts Mapped

OScope acts as a real-time laboratory for fundamental Operating System concepts covered in university syllabi:

10.1 The User-Space to Kernel-Space Transition (Ring 3 to Ring 0)

Unprivileged user programs run in CPU Ring 3. When an application needs hardware access (e.g. reading a disk file), it loads syscall arguments into CPU registers (`%rdi` , `%rsi` , `%rdx` , `%r10` , `%r8` , `%r9`) and the syscall number into `%rax` , then fires the `syscall` assembly instruction. The CPU hardware switches to Ring 0, transitions to the kernel stack, and looks up the handler in the **System Call Table (sys_call_table)**.

10.2 The `ptrace(2)` Mechanism & Circular Deadlocks

> **Critical Academic Finding: Circular ptrace Deadlock**
>
> When process A (OScope) traces process B, the Linux kernel intercepts B at every syscall entrance and exit, issuing `SIGTRAP` . B is halted until A reads the event and calls `PTRACE_SYSCALL` .
>
> **The Deadlock Scenario:** If process A attaches `strace` to process A itself:
>
> 1. Process A invokes a system call (e.g. reading from a pipe or drawing a Tkinter window).
> 2. The kernel immediately halts Process A.
> 3. `strace` receives the trap and prepares to resume A. But to do so, it must write to its output pipe.
> 4. Process A is frozen by the kernel, so it cannot read the pipe or service its GUI event loop!
> 5. **Result:** Both processes enter an unrecoverable circular wait deadlock. The GUI turns blank and ignores close buttons.
>
> **OScope's Solution:** Dynamically detects `os.getpid()` , flags it as `python3 [OScope Self]` , and programmatically disables self-tracing.

10.3 The Linux `/proc` Virtual Filesystem (procfs)

`/proc` is not a physical disk filesystem; it is a pseudo-filesystem generated dynamically by the Linux kernel memory manager and scheduler. Opening `/proc/[pid]/status` directly interrogates the kernel's internal `task_struct` and `mm_struct` structures:

- `VmRSS` (Resident Set Size): The exact portion of virtual memory currently mapped to physical RAM pages.
- `VmSize` : The entire virtual address space allocated to the process (including shared libraries and unmapped pages).
- `VmData` / `VmStk` : Memory consumption dedicated to the heap (data segment) and execution stack.
- `Process States` : `R` (Running/Runnable on run-queue), `S` (Interruptible Sleep waiting on an event/pipe), `D` (Uninterruptible Sleep waiting on disk I/O), `Z` (Zombie/Terminated child waiting for parent `waitpid`), `T` (Stopped by signal).

10.4 Linux Security: Yama LSM (`ptrace_scope`)

On modern Linux distributions and WSL2, the **Yama Linux Security Module (LSM)** enforces strict ptrace boundaries. By default, `kernel.yama.ptrace_scope = 1` prevents an unprivileged process from attaching `ptrace` to any process that is not its direct child, even if both processes share the same User ID (UID). OScope detects `Operation not permitted` and guides the user to set `echo 0 | sudo tee /proc/sys/kernel/yama/ptrace_scope` when attaching to pre-existing processes.

11. Implementation Timeline & Phase Milestones

OScope was developed following an iterative, test-driven engineering lifecycle:

Phase / Milestone	Key Deliverable	Technical Accomplishment
Core Engine	Real Linux Tracing	Integrated `strace` subprocess controller, POSIX process groups (`setpgrp`), raw terminal window.
Phase 1	Report Export	Engineered CSV and structured TXT export dialogs with statistical summaries and full session preservation.
Phase 2	Session Comparison	Built in-memory session persistence and side-by-side comparison dashboard with delta metrics (`+` / `-`).
Phase 3	Live Category Bar Chart	Implemented dynamic Tkinter Canvas rendering live 7-category frequency bars and percentage indicators.
Phase 4	Expanded Classification	Established 7-category taxonomy mapping 350+ system calls to core Operating System theoretical domains.
Phase 5	Syscall Argument Parsing	Created `detail_parser.py` to decode paths, flags, descriptors, buffer snippets, and return values.
Phase 6	Dedicated Failure Analysis	Isolated failed calls, integrated comprehensive errno dictionary, and plain-English error diagnostic explanations.
Phase 7	About This Syscall	Interactive knowledge base displaying man-page purpose, kernel transitions, and syllabus concept links.
Phase 8 & 9	Process Tree & Command History	Child process hierarchy tracking (`strace -f`), 1-click PID filtering, and in-memory dropdown command history.
Enhancement: CWD	Working Directory Selection	Custom directory selector with Windows-to-WSL path translation (`C:\...` -> `/mnt/c/...`) and default to `/home` .
Enhancement A	Live Process & Resource Dashboard	Linux `/proc` virtual filesystem collector, real-time CPU % calculation, memory segmentation, and sortable table.
Enhancement B & C	Attach to PID & Deadlock Guard	1-click `strace -p <PID>` attach with clean `SIGINT` detachment and circular self-tracing deadlock protection.

12. Feature Walkthrough & Graphical User Interface

12.1 Tab 1: Live Monitor

The command center of OScope. Users can type any Linux command (e.g. `ls -la`, `cat sample.txt`, `python3 -c "..."`) or select a target PID. Features include:

- **Working Directory Selector:** Run commands in any local folder, defaulting to `/home` for native Linux navigation.
- **Live Trace Table:** Virtualized table displaying real-time syscall stream with columns `#`, `PID`, `Syscall Name`, `Category`, `Key Details`, `Arguments`, and `Return Value`.
- **Live Search & Category Filter:** Instantaneous search filtering across syscall names and arguments.
- **Raw Terminal Window:** Independent dark-themed monospace window showing live raw strace text with auto-scrolling.

12.2 Tab 2: Processes & Resources (Linux `/proc` Engine)

Provides a macroscopic, real-time view of all running processes in the Linux system without third-party dependencies:

- **Process Table:** Displays PID, Name, User, State, CPU %, Physical RSS (MB), Virtual Memory VmSize (MB), Threads, and Command.
- **Interactive Sorting:** Click any header (CPU %, RSS, PID) to sort ascending or descending.
- **Deep Process Inspector:** Selecting any process displays its exact memory breakdown (Data Segment, Stack), execution state explanation, ancestor lineage chain (e.g. `systemd (1) → bash (892) → python3 (1420)`), and child processes.
- **Attach & Trace Syscalls Button:** Instantly transitions to the Live Monitor and attaches `strace -p <PID>` to inspect live syscalls of the selected process.

12.3 Tab 3: Process Tree Hierarchy (`strace -f`)

When tracing multi-process or multi-threaded programs (e.g. shells, compilers, web servers), the Process Tree tab builds an interactive hierarchical tree of parent and child processes. It records per-process syscall counters and execution status, allowing users to filter the entire live trace by any child PID with one click.

12.4 Tab 4: Live Category Bar Chart

Renders a dynamic, color-coded Tkinter Canvas bar chart showing the live distribution and percentage breakdown across all 7 OS categories. Updates smoothly in real time as syscalls are processed.

12.5 Tab 5: Failure Analysis & Errno Diagnostics

Filters out successful calls to focus exclusively on kernel rejections. Displays failure rates, errno distributions (e.g. `ENOENT`, `EACCES`), and human-readable explanations of why the operation failed.

12.6 Tab 6: Session History & Side-by-Side Comparison

Persists completed sessions in memory. Users can select any two sessions (e.g., comparing `cat sample.txt` vs `python3 gui.py`) to view numerical diffs (`+` / `-`) and top 10 syscall frequency comparisons.

13. Automated Verification & Testing Suite (`gui.py --test`)

To guarantee uncompromising software reliability, OScope includes a comprehensive, headless automated verification suite. It can be executed in any Linux or WSL2 terminal without manual GUI interaction:

```
wsl python3 gui.py --test
```

The test suite executes an end-to-end regression validation covering:

1. **Core Pipeline:** Spawning `strace` on sample commands (`cat sample.txt`), parsing tokens, categorizing syscalls, and verifying process group termination.
2. **CWD Execution:** Executing commands in custom working directories and confirming that `getcwd()` matches the target path in the trace.
3. **Procfs Engine:** Scanning `/proc`, confirming discovery of 50+ active processes, verifying memory parsing (RSS, VmData), and testing lineage resolution.
4. **PID Attachment & Clean Detach:** Spawning an independent background process, attaching `strace -p`, capturing syscall events, and issuing `SIGINT` to verify clean detachment without terminating the target process.
5. **Result:** 100% test pass rate across all phases.

14. Engineering Challenges, Edge Cases & Security Nuances

Challenge / Edge Case	Technical Root Cause	OScope Engineering Resolution
Paths With Spaces	Project directory contains spaces (e.g. `Operating Systems`). Passing unquoted paths to `shlex.split` fragments arguments.	Used `shlex.quote()` across all command construction strings in `tracer.py` and `gui.py`.
Circular ptrace Deadlock	Attaching `strace` to OScope's own PID halts the process on syscall entry, freezing the GUI event loop.	Detects `os.getpid()`, styles the row as `python3 [OScope Self]`, disables the attach button, and enforces popup guards.
Windows Console UTF-8	Windows PowerShell consoles default to `cp1252`, causing crash on unicode symbols (e.g. `\u2713`).	Reconfigured `sys.stdout` and `sys.stderr` to `utf-8` with `errors='replace'` and standard ASCII fallback markers.
Unfinished Syscalls	Blocking syscalls (e.g. `poll`, `epoll_wait`, `read`) output `<unfinished ...>` and resume on subsequent lines.	Implemented stateful reassembly buffer in `parser.py` to link split calls back into unified single records.
Yama LSM Restrictions	`kernel.yama.ptrace_scope = 1` blocks unprivileged ptrace of independent processes sharing the same UID.	Intercepts `Operation not permitted` and presents an educational dialog explaining the Yama LSM policy and remedy.

15. Future Scope & Academic Extensions

- **eBPF Kernel Probing Integration:** Complementing `ptrace` with eBPF kprobes/tracepoints for near-zero overhead production tracing.
- **Syscall Fault Injection:** Integrating `strace -e fault=...` to simulate disk failures and memory exhaustion for resilience testing.
- **Network Packet Correlation:** Correlating `sendto` and `recvfrom` buffer payloads with PCAP network packet captures.
- **Live Syscall Latency Heatmaps:** Visualizing execution latency distributions (nanoseconds to milliseconds) for kernel performance profiling.

16. Comprehensive Team Viva & Evaluator Defense Guide

This section prepares all 3 team members to confidently answer questions during the project evaluation and viva:

Q1. What is the fundamental difference between a library call and a system call?

Answer: A library call (e.g. `printf()`, `strcpy()`) executes entirely in unprivileged **User Space (Ring 3)** without switching CPU context. A system call (e.g. `write()`, `openat()`) triggers a hardware trap (`syscall` assembly instruction), switching the CPU to privileged **Kernel Space (Ring 0)** to request hardware services. `printf()` buffers output in user space and eventually invokes the `write()` system call.

Q2. How does OScope intercept system calls without modifying the target program's source code?

Answer: OScope leverages the Linux kernel's native `ptrace(2)` **(Process Trace)** system call via `strace`. `ptrace` allows a parent process to observe and control the execution of another process. When the traced process issues a syscall, the kernel halts it at the boundary, delivers a `SIGTRAP` signal to the tracer, allows inspection of CPU registers, and resumes execution.

Q3. Why did you implement both a /proc dashboard AND an strace monitor? Aren't they redundant?

Answer: No, they measure completely different dimensions of the Operating System! `/proc` is **Macroscopic and Sampling-Based**: it periodically reads the kernel's scheduler and memory structures (CPU %, RAM RSS, state), measuring *current resource state*. `strace` is **Microscopic and Event-Driven**: it intercepts every single user-to-kernel event (arguments, return codes, file descriptors). Combining both allows OScope to show what a process is consuming (procfs) and exactly what it is doing (syscalls).

Q4. What is a Zombie Process, and how does OScope detect it?

Answer: A Zombie process (State `Z`) is a process that has completed execution via `exit()`, but its entry remains in the kernel's Process Table because its parent has not yet read its exit status via `wait()` / `waitpid()`. OScope parses `/proc/[pid]/status` and flags state `Z` in the Processes & Resources dashboard with zero memory footprint.

Q5. Explain what a Circular ptrace Deadlock is and how OScope prevents it.

Answer: In Linux, if Process A attaches `strace` to Process A itself, every system call made by A causes the kernel to halt A. Because A is halted, it cannot service its Tkinter event loop, process X11 display events, or read `strace`'s output pipe. Meanwhile, `strace` is blocked waiting on A. This causes a permanent freeze. OScope prevents this by checking `os.getpid()`, styling its own row as `python3 [OScope Self]`, and programmatically disabling self-attachment.

Q6. What is the difference between VmRSS and VmSize in the memory profile?

Answer: VmSize represents the total virtual address space allocated to the process (including code, heap, stack, and mapped shared libraries), whether in RAM or on disk. **VmRSS (Resident Set Size)** is the actual physical RAM currently occupied by the process. VmSize is almost always significantly larger than VmRSS due to demand paging and virtual memory overcommit.

Q7. How does OScope ensure that stopping a trace doesn't kill an attached running program?

Answer: In `tracer.py`, OScope distinguishes between a spawned command and an attached PID. For a spawned command, it terminates the entire process group (`os.killpg`). For an attached PID, it sends `SIGINT` to `strace`. In Linux, `SIGINT` instructs `strace` to issue `ptrace(PTRACE_DETACH)`, allowing the target process to continue executing unhindered.

17. Conclusion & References

17.1 Conclusion

OScope successfully fulfills all technical, educational, and architectural goals for the 5th Semester Operating Systems Mini-Project. By unifying microscopic event-driven kernel tracing with macroscopic virtual filesystem resource inspection, OScope demystifies the user-to-kernel boundary. Its clean modularity, zero-dependency philosophy, responsive GUI, and 100% automated test coverage establish it as an exemplary, production-grade systems engineering project.

17.2 References

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). John Wiley & Sons.
2. Love, R. (2010). *Linux Kernel Development* (3rd ed.). Addison-Wesley Professional.
3. Kerrisk, M. (2010). *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. No Starch Press.
4. Linux Programmer's Manual: `strace(1)`, `ptrace(2)`, `proc(5)`, `syscalls(2)`, `signal(7)`.
5. Yama Linux Security Module Documentation: `Documentation/admin-guide/LSM/Yama.rst`.

End of Documentation — OScope Linux System Call Monitoring & Inspection Suite | SEM-5 OS Mini-Project